

RX Family

RSCAN Module Using Firmware Integration Technology

Introduction

This document describes the API for the RSCAN driver for the RX100, RX200 Series. Message transfers can be done using 1-message deep mailboxes, 4-message deep FIFOs, or any combination thereof.

NOTE: When developing an application with the E1 emulator, and the E1 emulator is powering the target board, be sure that it is supplying 5.0V and not 3.3V (specified in Debug Configuration) or the RSCAN will not operate properly.

Target Devices

The following is a list of devices that are currently supported by this API:

- RX140 Group (products with 128-Kbyte or larger ROM)
- RX231 Group
- RX23E-A Group
- RX23E-B Group
- RX23W Group
- RX24T Group (Chip version B)
- RX24U Group

When using this application note with other Renesas MCUs, careful evaluation is recommended after making modifications to comply with the alternate MCU.

Target Compilers

- Renesas Electronics C/C++ Compiler Package for RX Family
- GCC for Renesas RX
- IAR C/C++ Compiler for Renesas RX

For details of the confirmed operation contents of each compiler, refer to "6.1 Confirmed Operation Environment".

Contents

1. Overview	4
2. API Information.....	5
2.1 Hardware Requirements	5
2.2 Hardware Resource Requirements.....	5
2.3 Software Requirements	5
2.4 Limitations	5
2.5 Supported Toolchains	5
2.6 Header Files	5
2.7 Integer Types	5
2.8 Configuration Overview	6
2.9 Code Size	8
2.10 API Data Types	9
2.10.1 Box IDs (mailboxes and FIFOs)	9
2.10.2 R_CAN_Open() Data Types	9
2.10.3 Callback function events	10
2.10.4 R_CAN_InitChan() Data Types.....	10
2.10.5 R_CAN_ConfigFIFO() Data Types.....	10
2.10.6 R_CAN_AddRxRule() Data Types	11
2.10.7 R_CAN_SendMsg() Data Types	11
2.10.8 R_CAN_GetMsg() Data Types.....	11
2.10.9 R_CAN_GetHistoryEntry() Data Types	11
2.10.10 R_CAN_GetStatusMask() Data Types.....	12
2.10.11 R_CAN_GetCountErr() Data Types	13
2.10.12 R_CAN_Control() Data Types.....	13
2.11 Return Values	13
2.12 Adding the Module to Your Project	13
2.13 “for”, “while” and “do while” statements	15
3. API Functions	16
Summary	16
R_CAN_Open()	17
R_CAN_InitChan().....	20
R_CAN_ConfigFIFO()	23
R_CAN_AddRxRule().....	25
R_CAN_Control().....	27
R_CAN_SendMsg().....	29
R_CAN_GetMsg().....	31
R_CAN_GetHistoryEntry().....	32
R_CAN_GetStatusMask()	33

R_CAN_GetCountFIFO()	35
R_CAN_GetCountErr()	36
R_CAN_Close()	37
R_CAN_GetVersion()	38
4. Pin Setting	39
5. Demo Project	40
5.1 rscan_demo_rskrx231, rscan_demo_rskrx231_gcc	40
5.2 rscan_demo_rskrx24t, rscan_demo_rskrx24t_gcc	40
5.3 rscan_demo_rskrx24u, rscan_demo_rskrx24u_gcc	41
5.4 rscan_demo_rskrx140, rscan_demo_rskrx140_gcc	42
6. Appendices	43
6.1 Confirmed Operation Environment	43
6.2 Troubleshooting	50
Revision History	51

1. Overview

This driver is compatible with the RSCAN driver provided with the RZ/A1. Although the RSCAN peripheral on the RX100, RX200 Series is single channel, the API remains the same. A static configuration of mailboxes and FIFOs (boxes) is hardcoded as is done with the RZ/A1, but much fewer resources are available for use.

All mailboxes are one-message deep. There are 4 transmit mailboxes and 4 receive mailboxes. The transmit mailboxes can optionally be configured for interrupt operation, whereas the receive mailboxes cannot. The transmit mailboxes do not accept a message for transmit until the previous message has been sent. The receive mailboxes always contain the most recent message received, overwriting the previous contents without an error condition being generated. There is no hardware interrupt option available.

The transmit and receive FIFOs are 4-messages deep. FIFOs are used for the sending and receiving of messages just like a mailbox. These can optionally be configured to be interrupt driven. Setting a receive FIFO to interrupt on every message received would behave similar to a receive mailbox with interrupt support.

There is a special FIFO called the Transmit History FIFO, and this FIFO is 8 entries deep. The History FIFO logs all messages tagged in an `R_CAN_SendMsg()` call in the order they are sent. Note that any FIFO usage is optional and are not required for normal operation.

The RSCAN hardware processes all messages transmitted on the bus but uses Receive Rules to determine which messages to keep and which to ignore. A Receive Rule consists of two parts. The first part performs filtering on different parts of the message to see if the message should be kept. The second part specifies which box (receive mailbox or receive FIFO) to route the message to. After the hardware routes a message to a box, the function `R_CAN_GetMsg()` is used to read a message from the box.

There are two types of interrupts available- global interrupts and channel interrupts. The global interrupts indicate when a receive FIFO has received a message as well as when a global error occurs. These interrupts are enabled in the `r_rscan_rx_config.h` file. The driver detects the interrupt and calls a user callback function specified in `R_CAN_Open()` to process the particular event(s). The channel interrupts handle several transmit conditions as well as channel errors. These interrupts are also enabled in the `r_rscan_rx_config.h` file. The driver detects the interrupt and calls a user callback function specified in `R_CAN_InitChan()` to process the particular event(s).

By default, the following interrupts are enabled:

- RX, TX, or History FIFO threshold reached
- RX, TX, or History FIFO overflow occurred
- Channel entered Error Passive state
- Channel entered Bus Off state
- Channel recovered from Bus Off state

The following sequence of function calls is used to setup the CAN:

```
R_CAN_Open();  
R_CAN_InitChan(); // do for 1 channel  
R_CAN_ConfigFIFO(); // do for 0 or more FIFOs  
R_CAN_AddRxRule(); // do for 1-16 rules
```

Once the CAN is setup, the peripheral should enter normal communications mode or a test mode.

```
R_CAN_Control(); // Use CAN_CMD_SET_MODE_COMM or CAN_CMD_SET_MODE_TST_XXX
```

2. API Information

This Driver API follows the Renesas API naming standards.

2.1 Hardware Requirements

This driver utilizes the RSCAN peripheral.

2.2 Hardware Resource Requirements

In addition to the RSCAN peripheral, the driver requires:

- Two pins allocated for the CAN channel

2.3 Software Requirements

This driver is dependent upon the following FIT module:

- Renesas Board Support Package (r_bsp) v5.20 or higher

2.4 Limitations

Not all features of the peripheral are utilized. These include:

- Configurable depth transmit and receive FIFOs (all fixed at 4 instead of configurable 1 to 16)
- Transmit by message ID priority (will be done by mailbox number, 0 being highest priority)
- Transmit FIFO interval transmission
- Transmit mirroring
- Filter on mirrored messages
- DLC substitution
- Multiple destinations for each received message (will fix at 1 destination; could be up to 3)
- Different methods of Bus Off recovery (will be ISO11898-1 compliant)
- Forcible return from Bus Off
- Selection of protocol error flag accumulation vs first occurrence (will hard-code to accumulative for all channels)

2.5 Supported Toolchains

This driver has been confirmed to work with the toolchain listed in 6.1, Confirmed Operation Environment.

2.6 Header Files

All API calls and their supporting interface definitions are located in "r_rscan_rx_if.h".

Build-time configuration options are set in the file "r_rscan_rx_config.h"

Both of these files should be included by the user's application.

2.7 Integer Types

This project uses ANSI C99 "Exact width integer types" in order to make the code clearer and more portable. These types are defined in stdint.h.

2.8 Configuration Overview

Static configuration options for this driver are set by the user via the file `r_rscan_rx_config.h`.

Configuration options in <code>r_rscan_rx_config.h</code>		
Equate	Default Value	Description
<code>CAN_CFG_PARAM_CHECKING_ENABLE</code>	1	Setting to 0 removes parameter checking from the code. Setting to 1 includes parameter checking in the code.
<code>CAN_CFG_CLOCK_SOURCE</code>	0	If this equate is 0, the CAN clock source is $\frac{1}{2}$ the peripheral clock speed (<code>clk_c</code>). If this equate is 1, the source is the external CAN_CLOCK (<code>clk_xincan</code>).
<code>CAN_CFG_INT_PRIORITY</code>	5	Priority level for all CAN interrupts (0-31)
<code>CAN_CFG_INT_RXFIFO_THRESHOLD</code>	1	Setting to 0 disables interrupt when an RXFIFO threshold is reached. Setting to 1 enables interrupt. Requires FIFO to be initialized via <code>R_CAN_ConfigFIFO()</code> . <code>CAN_EVT_RXFIFO_THRESHOLD</code> is passed to the main callback function.
<code>CAN_CFG_INT_DLC_ERR</code>	0	Setting to 0 disables interrupt when a DLC error is detected. Setting to 1 enables interrupt. <code>CAN_EVT_GLOBAL_ERR</code> is passed to the main callback function.
<code>CAN_CFG_INT_FIFO_OVFL</code>	1	Setting to 0 disables interrupt when a TX, GW, or RX FIFO overflows. Setting to 1 enables interrupt. Requires FIFO to be initialized via <code>R_CAN_ConfigFIFO()</code> . <code>CAN_EVT_GLOBAL_ERR</code> is passed to the main callback function.
<code>CAN_CFG_INT_HIST_FIFO_OVFL</code>	1	Setting to 0 disables interrupt when a History FIFO overflows. Setting to 1 enables interrupt. Requires FIFO to be initialized via <code>R_CAN_ConfigFIFO()</code> . <code>CAN_EVT_GLOBAL_ERR</code> is passed to the main callback function.
<code>CAN_CFG_INT_TXFIFO_THRESHOLD</code>	1	Setting to 0 disables interrupt when a TXFIFO threshold is reached. Setting to 1 enables interrupt. Requires FIFO to be initialized via <code>R_CAN_ConfigFIFO()</code> . <code>CAN_EVT_TRANSMIT</code> is passed to the channel callback function.
<code>CAN_CFG_INT_HIST_FIFO_THRESHOLD</code>	1	Setting to 0 disables interrupt when the HIST_FIFO threshold is reached. Setting to 1 enables interrupt. Requires FIFO to be initialized via <code>R_CAN_ConfigFIFO()</code> . <code>CAN_EVT_TRANSMIT</code> is passed to the channel callback function.

Configuration options in <i>r_rscan_rx_config.h</i>		
Equate	Default Value	Description
<code>CAN_CFG_INT_MBX_TX_COMPLETE</code>	0	Setting to 0 disables interrupt when the mailbox completes transmission. Setting to 1 enables interrupt. CAN_EVT_TRANSMIT is passed to the channel callback function.
<code>CAN_CFG_INT_MBX_TX_ABORTED</code>	0	Setting to 0 disables interrupt when the mailbox transmit is aborted. Setting to 1 enables interrupt. CAN_EVT_TRANSMIT is passed to the channel callback function.
<code>CAN_CFG_INT_BUS_ERROR</code>	0	Setting to 0 disables interrupt when a bus error is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_INT_ERR_WARNING</code>	0	Setting to 0 disables interrupt when an error warning is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_INT_ERR_PASSIVE</code>	1	Setting to 0 disables interrupt when an error passive is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_INT_BUS_OFF_ENTRY</code>	1	Setting to 0 disables interrupt when a Bus Off error is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_INT_BUS_OFF_RECOVERY</code>	1	Setting to 0 disables interrupt when a Bus Off recovery is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_INT_OVERLOAD_FRAME_TX</code>	0	Setting to 0 disables interrupt when an overload is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_INT_BUS_LOCK</code>	0	Setting to 0 disables interrupt when a bus lock is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_INT_ARB_LOST</code>	0	Setting to 0 disables interrupt when arbitration loss is detected. Setting to 1 enables interrupt. CAN_EVT_CHANNEL_ERR is passed to the channel callback function.
<code>CAN_CFG_EN_NESTED_INT</code>	0	Specifies enable/disable of the nested interrupt. 0 : Disable nested interrupt. 1 : Enable nested interrupt.

Table 2.1: Info about the configuration

2.9 Code Size

The code size is based upon optimization level 2 for the RXC Toolchain. **These code sizes include all interrupt handlers (configured active or not), as well as all FIFO support code.**

The ROM (code and constants) and RAM (global data) sizes are determined by the build-time configuration options described in 2.8 Configuration Overview. The table lists reference values when the C compiler's compile options are set to their default values, as described in 2.5 Supported Toolchains. The compile option default values are optimization level: 2, optimization type: for size, and data endianness: little-endian. The code size varies depending on the C compiler version and compile options.

ROM, RAM and Stack Code Sizes							
Device	Category	Memory Used					
		Renesas Compiler		GCC		IAR Compiler	
		With Parameter Checking	Without Parameter Checking	With Parameter Checking	Without Parameter Checking	With Parameter Checking	Without Parameter Checking
RX231	ROM	3211 bytes	2683 bytes	5824 bytes	4976 bytes	5547 bytes	4916 bytes
	RAM	20 bytes	20 bytes	20 bytes	20 bytes	20 bytes	20 bytes
	STACK	36 bytes	36 bytes	-	-	144 bytes	144 bytes
RX23W	ROM	3211 bytes	2683 bytes	-	-	-	-
	RAM	20 bytes	20 bytes	-	-	-	-
	STACK	72 bytes	72 bytes	-	-	-	-
RX23E-A	ROM	3237 bytes	2723 bytes	6104 bytes	5232 bytes	5421 bytes	4782 bytes
	RAM	20 bytes	20 bytes	20 bytes	20 bytes	24 bytes	24 bytes
	STACK	100 bytes	100 bytes	-	-	96 bytes	96 bytes
RX140 (products with 128-Kbyte or larger ROM)	ROM	3204 bytes	2710 bytes	6088 bytes	5208 bytes	5296 bytes	4661 bytes
	RAM	20 bytes	20 bytes	0 bytes	0 bytes	20 bytes	20 bytes
	STACK	68 bytes	68 bytes	-	-	124 bytes	124 bytes
RX23E-B	ROM	3305 bytes	2811 bytes	4008 bytes	3288 bytes	5136 bytes	4501 bytes
	RAM	20 bytes	20 bytes	20 bytes	20 bytes	20 bytes	20 bytes
	STACK	68 bytes	68 bytes	-	-	96 bytes	96 bytes

2.10 API Data Types

This section details the data types that are used with the driver's API functions.

2.10.1 Box IDs (mailboxes and FIFOs)

```
typedef enum e_can_box
{
    CAN_BOX_NONE                = 0,    // unused parameter value

    CAN_BOX_CH0_TXMBX_0         = (CAN_FLG_TXMBX | 0),
    CAN_BOX_CH0_TXMBX_1         = (CAN_FLG_TXMBX | 1),
    CAN_BOX_CH0_TXMBX_2         = (CAN_FLG_TXMBX | 2),
    CAN_BOX_CH0_TXMBX_3         = (CAN_FLG_TXMBX | 3),

    CAN_BOX_RXMBX_0             = (CAN_FLG_RXMBX | 0),
    CAN_BOX_RXMBX_1             = (CAN_FLG_RXMBX | 1),
    CAN_BOX_RXMBX_2             = (CAN_FLG_RXMBX | 2),
    CAN_BOX_RXMBX_3             = (CAN_FLG_RXMBX | 3),

    CAN_BOX_RXFIFO_0            = (CAN_FLG_FIFO | CAN_MASK_RXFIFO_0),
    CAN_BOX_RXFIFO_1            = (CAN_FLG_FIFO | CAN_MASK_RXFIFO_1),

    CAN_BOX_TXFIFO              = (CAN_FLG_FIFO | CAN_MASK_CH0_TXFIFO_0),

    CAN_BOX_HIST_FIFO           = (CAN_FLG_FIFO | CAN_MASK_CH0_HIST_FIFO),
} can_box_t;
```

2.10.2 R_CAN_Open() Data Types

```
typedef enum e_can_timestamp_src
{
    CAN_TIMESTAMP_SRC_HALF_PCLK = 0,
    CAN_TIMESTAMP_SRC_CANMCLK   = 1,    // obtained from EXTAL pin
    CAN_TIMESTAMP_SRC_END_ENUM
} can_timestamp_src_t;

typedef enum e_can_timestamp_div
{
    CAN_TIMESTAMP_DIV_1         = 0,
    CAN_TIMESTAMP_DIV_2         = 1,
    CAN_TIMESTAMP_DIV_4         = 2,
    CAN_TIMESTAMP_DIV_8         = 3,
    CAN_TIMESTAMP_DIV_16        = 4,
    CAN_TIMESTAMP_DIV_32        = 5,
    CAN_TIMESTAMP_DIV_64        = 6,
    CAN_TIMESTAMP_DIV_128       = 7,
    CAN_TIMESTAMP_DIV_256       = 8,
    CAN_TIMESTAMP_DIV_512       = 9,
    CAN_TIMESTAMP_DIV_1024      = 10,
    CAN_TIMESTAMP_DIV_2048      = 11,
    CAN_TIMESTAMP_DIV_4096      = 12,
    CAN_TIMESTAMP_DIV_8192      = 13,
    CAN_TIMESTAMP_DIV_16384     = 14,
    CAN_TIMESTAMP_DIV_32768     = 15,
    CAN_TIMESTAMP_DIV_END_ENUM
} can_timestamp_div_t;

typedef struct st_can_cfg
{
    can_timestamp_src_t    timestamp_src;
    can_timestamp_div_t    timestamp_div;
} can_cfg_t;
```

2.10.3 Callback function events

```
typedef enum e_can_cb_evt          // callback function events
{
    // Main Callback Events
    CAN_EVT_RXFIFO_THRESHOLD,      // RX FIFO threshold
    CAN_EVT_GLOBAL_ERR,           // RX or Hist FIFO overflow, or DLC error

    // Channel Callback Events
    CAN_EVT_TRANSMIT,              // mbx tx complete or aborted,
                                   // tx or history FIFO threshold
    CAN_EVT_CHANNEL_ERR,

} can_cb_evt_t;
```

2.10.4 R_CAN_InitChan() Data Types

```
typedef struct st_can_bitrate
{
    uint16_t    prescaler;        // 1-1024
    uint8_t      tseg1;           // 4-16
    uint8_t      tseg2;           // 2-8
    uint8_t      sjw;             // 1-4
} can_bitrate_t;

/* Sample settings for 500kbps */
#define CAN_RSK_27MHZ_PCLKB_500KBPS_PRESCALER    3
#define CAN_RSK_27MHZ_PCLKB_500KBPS_TSEG1        5
#define CAN_RSK_27MHZ_PCLKB_500KBPS_TSEG2        3
#define CAN_RSK_27MHZ_PCLKB_500KBPS_SJW          1

#define CAN_RSK_32MHZ_PCLKB_500KBPS_PRESCALER    2
#define CAN_RSK_32MHZ_PCLKB_500KBPS_TSEG1        11
#define CAN_RSK_32MHZ_PCLKB_500KBPS_TSEG2        4
#define CAN_RSK_32MHZ_PCLKB_500KBPS_SJW          4

// alternate settings
#define CAN_RSK_8MHZ_XTAL_500KBPS_PRESCALER      1 // 2
#define CAN_RSK_8MHZ_XTAL_500KBPS_TSEG1         10 // 5
#define CAN_RSK_8MHZ_XTAL_500KBPS_TSEG2         5 // 2
#define CAN_RSK_8MHZ_XTAL_500KBPS_SJW           1 // 1
```

2.10.5 R_CAN_ConfigFIFO() Data Types

```
typedef enum e_can_fifo_threshold // NOTE: History FIFO can only have a
{                                  // threshold of 1 or 6
    CAN_FIFO_THRESHOLD_1 = 1,      // every message
    CAN_FIFO_THRESHOLD_2 = 3,      // 4/8 of 4
    CAN_FIFO_THRESHOLD_3 = 5,      // 6/8 of 4
    CAN_FIFO_THRESHOLD_6 = 6,      // History FIFO Only!
    CAN_FIFO_THRESHOLD_FULL = 7,    // 8/8 of 4
    CAN_FIFO_THRESHOLD_END_ENUM
} can_fifo_threshold_t;
```

2.10.6 R_CAN_AddRxRule() Data Types

```
typedef struct st_can_filter
{
    uint8_t      check_ide:1;
    uint8_t      ide;
    uint8_t      check_rtr:1;
    uint8_t      rtr;
    uint32_t     id;
    uint32_t     id_mask;
    uint8_t      min_dlc;
    uint16_t     label;           // 12-bit label
} can_filter_t;
```

2.10.7 R_CAN_SendMsg() Data Types

```
typedef struct st_can_txmsg
{
    uint8_t      ide;
    uint8_t      rtr;
    uint32_t     id;
    uint8_t      dlc;
    uint8_t      data[8];
    bool_t       one_shot;        // no retries on error; txmbx only
    bool_t       log_history;     // true if want to log
    uint8_t      label;           // 8-bit label for History FIFO
} can_txmsg_t;
```

2.10.8 R_CAN_GetMsg() Data Types

```
typedef struct st_can_rxmsg
{
    uint8_t      ide;
    uint8_t      rtr;
    uint32_t     id;
    uint8_t      dlc;
    uint8_t      data[8];
    uint16_t     label;           // 12-bit label from receive rule
    uint16_t     timestamp;
} can_rxmsg_t;
```

2.10.9 R_CAN_GetHistoryEntry() Data Types

```
typedef struct st_can_history
{
    can_box_t    box_id;          // box which sent message
    uint8_t      label;           // associated 8-bit label
} can_history_t;
```

2.10.10 R_CAN_GetStatusMask() Data Types

```

typedef enum e_can_stat
{
    CAN_STAT_FIFO_EMPTY,
    CAN_STAT_FIFO_THRESHOLD,
    CAN_STAT_FIFO_OVFL,           // bits reset after reading
    CAN_STAT_RXMBX_FULL,
    CAN_STAT_GLOBAL_ERR,         // DLC error bit is reset after reading
    CAN_STAT_CH_TXMBX_SENT,      // bits reset after reading
    CAN_STAT_CH_TXMBX_ABORTED,   // bits reset after reading
    CAN_STAT_CH_ERROR,           // bits reset after reading
    CAN_STAT_END_ENUM
} can_stat_t;

/* Returned mask values (multiple bits may be set at the same time)

/* CAN_STAT_CH_TXMBX_SENT, CAN_STAT_CH_TXMBX_ABORTED */
#define CAN_MASK_TXMBX_0          (0x0001)
#define CAN_MASK_TXMBX_1          (0x0002)
#define CAN_MASK_TXMBX_2          (0x0004)
#define CAN_MASK_TXMBX_3          (0x0008)

/* CAN_STAT_RXMBX_FULL */
#define CAN_MASK_RXMBX_0          (0x0001)
#define CAN_MASK_RXMBX_1          (0x0002)
#define CAN_MASK_RXMBX_2          (0x0004)
#define CAN_MASK_RXMBX_3          (0x0008)

/* CAN_STAT_FIFO_EMPTY, CAN_STAT_FIFO_THRESHOLD, CAN_STAT_FIFO_OVFL */
#define CAN_MASK_RXFIFO_0          (0x00000001)
#define CAN_MASK_RXFIFO_1          (0x00000002)
#define CAN_MASK_TXFIFO           (0x00000100)
#define CAN_MASK_HIST_FIFO        (0x00800000)

/* CAN_STAT_GLOBAL_ERR */
#define CAN_MASK_ERR_DLC           (0x0001)
#define CAN_MASK_ERR_RX_OVFL       (0x0002)
#define CAN_MASK_ERR_HIST_OVFL     (0x0004)
#define CAN_MASK_ERR_FIFO_OVFL     (0x0006)

/* CAN_STAT_CH_ERROR */
#define CAN_MASK_ERR_PROTOCOL      (0x0001)
#define CAN_MASK_ERR_WARNING       (0x0002)
#define CAN_MASK_ERR_PASSIVE       (0x0004)
#define CAN_MASK_ERR_BUS_OFF_ENTRY (0x0008)
#define CAN_MASK_ERR_BUS_OFF_EXIT  (0x0010)
#define CAN_MASK_ERR_OVERLOAD      (0x0020)
#define CAN_MASK_ERR_DOMINANT_LOCK (0x0040)
#define CAN_MASK_ERR_ARB_LOST      (0x0080)
#define CAN_MASK_ERR_STUFF          (0x0100)
#define CAN_MASK_ERR_FORM           (0x0200)
#define CAN_MASK_ERR_ACK            (0x0400)
#define CAN_MASK_ERR_CRC            (0x0800)
#define CAN_MASK_ERR_RECESSIVE_BIT  (0x1000)
#define CAN_MASK_ERR_DOMINANT_BIT   (0x2000)
#define CAN_MASK_ERR_ACK_DELIMITER (0x4000)

```

2.10.11 R_CAN_GetCountErr() Data Types

```
typedef enum e_can_count
{
    CAN_COUNT_RX_ERR,
    CAN_COUNT_TX_ERR,
    CAN_COUNT_END_ENUM
} can_count_t;
```

2.10.12 R_CAN_Control() Data Types

```
typedef enum e_can_cmd
{
    CAN_CMD_ABORT_TX, // argument: transmit mailbox id
    CAN_CMD_RESET_TIMESTAMP,
    CAN_CMD_SET_MODE_COMM, // start normal bus communications
    CAN_CMD_SET_MODE_TST_STANDARD,
    CAN_CMD_SET_MODE_TST_LISTEN,
    CAN_CMD_SET_MODE_TST_EXT_LOOPBACK,
    CAN_CMD_SET_MODE_TST_INT_LOOPBACK,
    CAN_CMD_END_ENUM
} can_cmd_t;
```

2.11 Return Values

API function return values. This enum is found in `r_rscan_rx_if.h` along with the API function declarations.

```
typedef enum e_can_err // CAN API error codes
{
    CAN_SUCCESS=0,
    CAN_ERR_OPENED, // Call to Open already made
    CAN_ERR_NOT_OPENED, // Call to Open not yet made
    CAN_ERR_INIT_DONE, // Call to InitChan already made for channel
    CAN_ERR_CH_NO_INIT, // Channel not initialized
    CAN_ERR_INVALID_ARG, // Invalid argument passed to function
    CAN_ERR_MISSING_CALLBACK, // Callback func not provided and ints requested
    CAN_ERR_MAX_RULES, // Max configured rules already present
    CAN_ERR_BOX_FULL, // Transmit mailbox or FIFO is full
    CAN_ERR_BOX_EMPTY, // Receive mailbox or FIFO is full
    CAN_ERR_ILLEGAL_MODE, // Not in proper mode for request
    CAN_ERR_TIME_OUT // Time Out error
} can_err_t;
```

2.12 Adding the Module to Your Project

This module must be added to each project in which it is used. Renesas recommends the method using the Smart Configurator described in (1) or (2) below. However, the Smart Configurator only supports some RX devices. Please use the methods of (3) for RX devices that are not supported by the Smart Configurator.

- (1) Adding the FIT module to your project using the Smart Configurator in e² studio
By using the Smart Configurator in e² studio, the FIT module is automatically added to your project. Refer to “Renesas e² studio Smart Configurator User Guide (R20AN0451)” for details.
- (2) Adding the FIT module to your project using the Smart Configurator in CS+
By using the Smart Configurator Standalone version in CS+, the FIT module is automatically added to your project. Refer to “Renesas e² studio Smart Configurator User Guide (R20AN0451)” for details.

- (3) Adding the FIT module to your project in CS+
In CS+, please manually add the FIT module to your project. Refer to “Adding Firmware Integration Technology Modules to CS+ Projects (R01AN1826)” for details.

2.13 “for”, “while” and “do while” statements

In this module, “for”, “while” and “do while” statements (loop processing) are used in processing to wait for register to be reflected and so on. For these loop processing, comments with “WAIT_LOOP” as a keyword are described. Therefore, if user incorporates fail-safe processing into loop processing, user can search the corresponding processing with “WAIT_LOOP”.

The following shows example of description.

while statement example:

```
/* WAIT_LOOP */
while(0 == SYSTEM.OSCOVFSR.BIT.PLOVF)
{
    /* The delay period needed is to make sure that the PLL has stabilized. */
}
```

for statement example:

```
/* Initialize reference counters to 0. */
/* WAIT_LOOP */
for (i = 0; i < BSP_REG_PROTECT_TOTAL_ITEMS; i++)
{
    g_protect_counters[i] = 0;
}
```

do while statement example:

```
/* Reset completion waiting */
do
{
    reg = phy_read(ether_channel, PHY_REG_CONTROL);
    count++;
} while ((reg & PHY_CONTROL_RESET) && (count < ETHER_CFG_PHY_DELAY_RESET)); /* WAIT_LOOP */
```

3. API Functions

Summary

The following functions are included in this design:

Function	Description
R_CAN_Open()	Initializes the driver's internal structures and all of the receive mailboxes.
R_CAN_InitChan()	Sets the bit rate clock for the channel and initializes all of the transmit mailboxes.
R_CAN_ConfigFIFO()	Initializes a FIFO for usage. This function should not be called if FIFOs are not used.
R_CAN_AddRxRule()	Adds a receive rule to a channel. Specifies receive message filter and destination routing.
R_CAN_SendMsg()	Loads a message into a transmit mailbox or FIFO for transmission.
R_CAN_GetMsg()	Fetches a message from a receive mailbox or FIFO.
R_CAN_GetHistoryEntry()	Fetches a log entry from a transmit history FIFO.
R_CAN_GetStatusMask()	Returns a 32-bit mask based upon the status requested. Bit #defines have the form CAN_MASK_XXX.
R_CAN_GetCountFIFO()	Returns the number of messages in a FIFO.
R_CAN_GetCountErr()	Returns the number of transmit or receive errors.
R_CAN_Control()	Handles special operations and mode changes.
R_CAN_Close()	Removes power to the CAN peripheral and disables the associated interrupts.
R_CAN_GetVersion()	Returns the driver version number.

R_CAN_Open()

This function initializes the driver's internal structures and all of the receive mailboxes.

Format

```
can_err_t R_CAN_Open(can_cfg_t *p_cfg,
                    void (* const p_callback) (can_cb_evt_t event,
                    void *p_args));
```

Parameters

p_cfg

Pointer to configuration structure. The element type definitions are provided in Section 2.10.1.

```
typedef struct st_can_cfg
{
    can_timestamp_src_t    ts_source;
    can_timestamp_div_t    ts_divisor;
} can_cfg_t;
```

p_callback

Optional pointer to main callback function. Must be present if interrupts are enabled in r_rscan_rx_config.h for RX FIFOs or global errors

event

First parameter for callback function. Specifies the interrupt source (see Section 2.10.3)

p_args

Second parameter for callback function (unused).

Return Values

CAN_SUCCESS:

Successful

CAN_ERR_OPENED:

Call to Open already made

CAN_ERR_INVALID_ARG:

An element of the p_cfg structure contains an invalid value.

CAN_ERR_MISSING_CALLBACK:

A callback function was not provided and

a main callback interrupt is enabled in config.h

CAN_ERR_TIME_OUT:

Time Out error

Properties

Prototyped in file "r_rscan_rx_if.h"

Description

This function initializes the driver's internal structures, applies clock to the peripheral, and sets the Global and Channel Modes to Reset. The timestamp is configured as per the p_cfg argument, and all receive mailboxes are initialized.

If interrupts are enabled in r_rscan_rx_config.h for receive FIFO thresholds, or DLC or FIFO overflow errors, a callback function must be provided here. Otherwise, NULL is entered.

Reentrant

No.

Example: Polling Configuration

```

/* All main callback interrupt sources are set to 0 in r_rscan_rx_config.h
*/

can_cfg_t    config;
can_err_t    err;

/* Configure timestamp and Open driver */
config.timestamp_src = CAN_TIMESTAMP_SRC_HALF_PCLK;
config.timestamp_div = CAN_TIMESTAMP_DIV_1024;
err = R_CAN_Open(&config, NULL);

```

Example: Interrupt Configuration

```

/* 1+ main callback interrupt sources are set to 1 in r_rscan_rx_config.h */

can_cfg_t    config;
can_err_t    err;

/* Configure timestamp and Open driver */
config.timestamp_src = CAN_TIMESTAMP_SRC_HALF_PCLK;
config.timestamp_div = CAN_TIMESTAMP_DIV_1024;
err = R_CAN_Open(&config, MyCallback);

/* Sample callback function */
void MyCallback(can_cb_evt_t event, void *p_args)
{
    uint32_t    mask;
    can_err_t    err;

    if (event == CAN_EVT_RXFIFO_THRESHOLD)
    {
        mask = R_CAN_GetStatusMask(CAN_STAT_FIFO_THRESHOLD, NULL, &err);

        /* check RXFIFOs in use */
        if (mask & CAN_MASK_RXFIFO_1)
        {
            /* read messages */
        }
    }
    else if (event == CAN_EVT_GLOBAL_ERR)
    {
        mask = R_CAN_GetStatusMask(CAN_STAT_GLOBAL_ERR, NULL, &err);

        if (mask & CAN_MASK_ERR_DLC)
        {
            /* handle DLC error */
        }

        if (mask & CAN_MASK_ERR_FIFO_OVFL)
        {
            mask = R_CAN_GetStatusMask(CAN_STAT_FIFO_OVFL, NULL, &err);

            /* check the RXFIFOs, GWFIFO, and HIST_FIFOs in use */
            if (mask & CAN_MASK_HIST_FIFO)
            {
                /* handle error */
            }
        }
    }
}

```

Special Notes:

The ports pins used by the RSCAN peripheral should be initialized prior to calling R_CAN_Open(). Here are some examples:

RX231:

```
R_BSP_RegisterProtectDisable(BSP_REG_PROTECT_MPC);

PORT5.PODR.BIT.B5 = 1;
PORT5.PODR.BIT.B4 = 0;
MPC.P54PFS.BYTE = 0x10;    // Pin Func Select P54 CTXD0
MPC.P55PFS.BYTE = 0x10;    // Pin Func Select P55 CRXD0
PORT5.PDR.BIT.B4 = 1;      // set TX pin direction to output
PORT5.DSCR.BIT.B4 = 1;     // High-drive output
PORT5.PDR.BIT.B5 = 0;      // set RX pin direction to input (dflt)
PORT5.PMR.BIT.B4 = 1;      // set TX pin mode to peripheral
PORT5.PMR.BIT.B5 = 1;      // set RX pin mode to peripheral

R_BSP_RegisterProtectEnable(BSP_REG_PROTECT_MPC);
```

RX24T:

```
R_BSP_RegisterProtectDisable(BSP_REG_PROTECT_MPC);

PORTA.PODR.BIT.B1 = 1;
PORTA.PODR.BIT.B0 = 0;
MPC.PA0PFS.BYTE = 0x10;    // Pin Func Select PA0 CTXD0
MPC.PA1PFS.BYTE = 0x10;    // Pin Func Select PA1 CRXD0
PORTA.PDR.BIT.B0 = 1;      // set TX pin direction to output
PORTA.DSCR.BIT.B0 = 1;     // High-drive output
PORTA.PDR.BIT.B1 = 0;      // set RX pin direction to input (dflt)
PORTA.PMR.BIT.B0 = 1;      // set TX pin mode to peripheral
PORTA.PMR.BIT.B1 = 1;      // set RX pin mode to peripheral

R_BSP_RegisterProtectEnable(BSP_REG_PROTECT_MPC);
```

RX24U:

```
R_BSP_RegisterProtectDisable(BSP_REG_PROTECT_MPC);

PORTF.PODR.BIT.B3 = 1;
PORTF.PODR.BIT.B2 = 0;
MPC.PF2PFS.BYTE = 0x10;    // Pin Func Select PF2 CTXD0
MPC.PF3PFS.BYTE = 0x10;    // Pin Func Select PF3 CRXD0
PORTF.PDR.BIT.B2 = 1;      // set TX pin direction to output
PORTF.DSCR.BIT.B2 = 1;     // High-drive output
PORTF.PDR.BIT.B3 = 0;      // set RX pin direction to input (dflt)
PORTF.PMR.BIT.B2 = 1;      // set TX pin mode to peripheral
PORTF.PMR.BIT.B3 = 1;      // set RX pin mode to peripheral

R_BSP_RegisterProtectEnable(BSP_REG_PROTECT_MPC);
```

R_CAN_InitChan()

This function sets the bit rate clock for the channel and initializes all of the transmit mailboxes.

Format

```
can_err_t R_CAN_InitChan(uint8_t chan,
                        can_bitrate_t *p_baud,
                        void (* const p_chcallback) (uint8_t chan,
                                                    can_cb_evt_t event,
                                                    void *p_args));
```

Parameters

chan

Channel to initialize (0 is only valid value).

p_baud

Pointer to bit rate structure. See the “Bit Timing Setting” section under CAN Module in the Hardware Manual for calculating settings. Some default values are provided in `r_rscan_rx_if.h`.

```
typedef struct st_can_bitrate
{
    uint16_t    prescaler;
    uint8_t     tseg1;
    uint8_t     tseg2;
    uint8_t     sjw;
} can_bitrate_t;
```

p_chcallback

Optional pointer to channel callback function. Must be present if interrupts are enabled in `r_rscan_rx_config.h` for TX mailboxes, TX FIFOs, History FIFOs, or bus errors.

channel

First parameter for channel callback function. Specifies the channel interrupt occurred on (always 0).

event

Second parameter for channel callback function. Specifies the interrupt source (see Section 2.10.3)

p_args

Third parameter for callback function (unused).

Return Values

`CAN_SUCCESS`:

Successful

`CAN_ERR_ILLEGAL_MODE`:

Not in global reset mode (results from call to Open())

`CAN_ERR_INVALID_ARG`:

An invalid argument was provided

`CAN_ERR_MISSING_CALLBACK`:

A callback function was not provided and a channel interrupt is enabled in config.h

Properties

Prototyped in file “`r_rscan_rx_if.h`”

Description

This function initializes all of the channel’s transmit mailboxes, sets the bit rate, and enables interrupt sources for the channel as specified in the `r_rscan_rx_config.h` file. Default values for *p_baud* are provided in `r_rscan_rx_if.h`.

If interrupts are enabled in `r_rscan_rx_config.h` for TX mailboxes, TX FIFOs, History FIFOs, or bus errors, a callback function must be provided here. Otherwise, NULL is entered.

Reentrant

Yes, for different channels.

Example: Polling Configuration

```

/* All channel interrupt sources are set to 0 in r_rscan_rx_config.h */

can_bitrate_t    baud;
can_err_t        err;

/* Initialize channel 0 for RSKRX231 */
baud.prescaler = CAN_RSK_8MHZ_XTAL_500KBPS_PRESCALER;
baud.tseg1 = CAN_RSK_8MHZ_XTAL_500KBPS_TSEG1;
baud.tseg2 = CAN_RSK_8MHZ_XTAL_500KBPS_TSEG2;
baud.sjw = CAN_RSK_8MHZ_XTAL_500KBPS_SJW;

err = R_CAN_InitChan(CAN_CH0, &baud, NULL);

```

Example: Interrupt Configuration

```

/* 1+ channel interrupt sources are set to 1 in r_rscan_rx_config.h */

can_bitrate_t    baud;
can_err_t        err;

/* Initialize channel 0 for RSKRX231 */
baud.prescaler = CAN_RSK_8MHZ_XTAL_500KBPS_PRESCALER;
baud.tseg1 = CAN_RSK_8MHZ_XTAL_500KPS_TSEG1;
baud.tseg2 = CAN_RSK_8MHZ_XTAL_500KPS_TSEG2;
baud.sjw = CAN_RSK_8MHZ_XTAL_500KPS_SJW;

err = R_CAN_InitChan(CAN_CH0, &baud, MyChanCallback);

/* Sample callback function template */
void MyChanCallback(uint8_t      chan,
                    can_cb_evt_t event,
                    void          *p_args)
{
    uint32_t  mask;
    can_err_t err;

    if (event == CAN_EVT_TRANSMIT)
    {
        mask = R_CAN_GetStatusMask(CAN_STAT_CH_TXMBX_SENT, chan, &err);

        /* check transmit mailboxes in use */
        if (mask & CAN_MASK_TXMBX_3)
        {
            /* do stuff */
        }

        mask = R_CAN_GetStatusMask(CAN_STAT_CH_TXMBX_ABORTED, chan, &err);

        /* check transmit mailboxes in use */
        if (mask & CAN_MASK_TXMBX_0)
        {
            /* do stuff */
        }

        mask = R_CAN_GetStatusMask(CAN_STAT_FIFO_THRESHOLD, NULL, &err);

        /* check transmit and history FIFOs in use */
        if (mask & CAN_MASK_TXFIFO)
        {
            /* load next batch of messages for transmit */
        }
    }
}

```

```
}  
  
else if (event == CAN_EVT_CHANNEL_ERR)  
{  
    mask = R_CAN_GetStatusMask(CAN_STAT_CH_ERROR, chan, &err);  
  
    /* check individual errors if desired */  
    if (mask & CAN_MASK_ERR_BUS_OFF_ENTRY)  
    {  
        /* handle error */  
    }  
  
    if (mask & CAN_MASK_ERR_BUS_OFF_EXIT)  
    {  
        /* handle recovery */  
    }  
}  
}
```

Special Notes:

None.

R_CAN_ConfigFIFO()

This function initializes a FIFO for usage. This function should not be called if FIFOs are not used.

Format

```
can_err_t R_CAN_ConfigFIFO(can_box_t      fifo,
                           can_fifo_threshold_t threshold,
                           can_box_t      txmbx_id);
```

Parameters

fifo_id

Box id for FIFO (see Section 2.10.1)

threshold

Number of messages needed in FIFO to set interrupt flag (see Section 2.10.5). Note that the only valid thresholds for the History FIFOs is 1 or 6 entries. All others may use 1, 2, 3, or full (4).

txmbx_id

Box id for associated transmit mailbox (for transmit FIFOs only). This argument is ignored for receive and history FIFOs.

Return Values

CAN_SUCCESS:

Successful

CAN_ERR_ILLEGAL_MODE:

Not in global reset mode (results from call to Open())

CAN_ERR_CH_NO_INIT:

Channel not initialized yet

CAN_ERR_INVALID_ARG:

An invalid argument was provided

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

FIFO usage is optional.

This function is used to activate a FIFO. Transmit and receive FIFOs are 4 entries deep (history FIFO is 8 deep). The transmit FIFO must have associated with it a standard transmit mailbox. The number of the mailbox determines the priority of the FIFO when transmitting (mailbox 0 = highest priority; mailbox 3 = lowest).

Reentrant

Yes, for different FIFOs.

Example: RX FIFO

```
can_err_t      err;

/*
 * Set interrupt flag on every message received on RX FIFO 0.
 * Interrupt occurs if CAN_CFG_INT_RXFIFO_THRESHOLD is set to 1 in config.h.
 * Interrupt calls main callback function with CAN_EVT_RXFIFO_THRESHOLD.
 */
err = R_CAN_ConfigFIFO(CAN_BOX_RXFIFO_0,
                      CAN_FIFO_THRESHOLD_1,
                      CAN_BOX_NONE);           // unused field here
```

Example: TX FIFO

```
can_err_t      err;

/*
 * Associate mailbox 3 with TX FIFO 0 on channel 0.
```

```
/* Set interrupt flag when 2 messages remain in FIFO.
 * Interrupt occurs if CAN_CFG_INT_TXFIFO_THRESHOLD is set to 1 in config.h.
 * Interrupt calls channel callback function with CAN_EVT_TRANSMIT.
 */
err = R_CAN_ConfigFIFO(CAN_BOX_TXFIFO,
                      CAN_FIFO_THRESHOLD_2,
                      CAN_BOX_CH0_TXMBX_3);
```

Example: History FIFO

```
can_err_t      err;

/*
 * Set threshold to 6 for History FIFO.
 * Interrupt occurs if CAN_CFG_INT_HIST_FIFO_THRESHOLD is set to 1 in config.h.
 * Interrupt calls channel callback function with CAN_EVT_TRANSMIT.
 */
err = R_CAN_ConfigFIFO(CAN_BOX_HIST_FIFO,
                      CAN_FIFO_THRESHOLD_6,
                      CAN_BOX_NONE);           // unused field here
```

Special Notes:

None.

R_CAN_AddRxRule()

This function adds a receive rule to a channel. Specifies receive message filter and destination routing.

Format

```
can_err_t R_CAN_AddRxRule(uint8_t chan,
                          can_filter_t *p_filter,
                          can_box_t dst_box);
```

Parameters

chan

Channel to apply rule to (always 0)

p_filter

Pointer to rule information.

```
typedef struct st_can_filter
{
    uint8_t    check_ide:1;
    uint8_t    ide;
    uint8_t    check_rtr:1;
    uint8_t    rtr;
    uint32_t    id;
    uint32_t    id_mask;
    uint8_t    min_dlc;
    uint16_t    label;           // 12-bit label
} can_filter_t;
```

dst_box

Destination box (receive mailbox or receive FIFO) to route message to (see Section 2.10.1).

Return Values

CAN_SUCCESS:

Successful

CAN_ERR_ILLEGAL_MODE:

Not in global reset mode (results from call to Open())

CAN_ERR_CH_NO_INIT:

Channel not initialized yet

CAN_ERR_INVALID_ARG:

An invalid argument was provided

CAN_ERR_MAX_RULES:

Max rules already present (as defined in r_rscan_rx_config.h,

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

This function is used to add a receive rule to a channel. There are two parts to this. The first part is specifying a filter as to which fields to inspect on received messages. The second part is to specify a destination to route the message to if it passes the filter test.

A “1” in the *id_mask* field indicates that the corresponding bit in a received message ID will be checked against the bit in the *id* field in this filter (see Examples).

The *label* field in the rule is optional. It is associated with each message that passes the filter. This may serve as a quick identification of a message when it is fetched from a receive box (mailbox or FIFO) using R_CAN_GetMsg().

Reentrant

No.

Example 1: Match a range of messages

```
can_filter_t filter;
can_err_t err;

/* Setup filter */
filter.check_ide = 1; // check the IDE field in message
filter.ide = 0; // 11-bit ID
filter.check_rtr = 0; // do not check the RTR field in message
filter.rtr = 0; // (value does not matter here; not checking)
filter.id = 0x040; // message ID
filter.id_mask = 0x7F0; // messages with IDs of 0x040-0x04F are accepted
filter.min_dlc = 4; // message data must be at least four bytes long
filter.label = 0x800; // arbitrary label applied to msgs of this type

/* Add rule to channel 0. Route filtered messages to receive mailbox 3. */
err = R_CAN_AddRxRule(CAN_CH0, &filter, CAN_BOX_RXMBX_3);
```

Example 2: Exact match for message

```
can_filter_t filter;
can_err_t err;

/* Setup filter */
filter.check_ide = 1; // check the IDE field in message
filter.ide = 0; // 11-bit ID
filter.check_rtr = 0; // do not check the RTR field in message
filter.rtr = 0; // (value does not matter here; not checking)
filter.id = 0x040; // message ID
filter.id_mask = 0x7FF; // ID must match 0x040 exactly
filter.min_dlc = 6; // message data must be at least six bytes long
filter.label = 0x700; // arbitrary label applied to msgs of this type

/* Add rule to channel 0. Route filtered messages to receive mailbox 2. */
err = R_CAN_AddRxRule(CAN_CH0, &filter, CAN_BOX_RXMBX_2);
```

Special Notes:

Rules cannot be entered after entering communications mode.

R_CAN_Control()

This function handles special operations and mode changes.

Format

```
can_err_t R_CAN_Control(can_cmd_t cmd,
                        uint32_t arg1);
```

Parameters

cmd

Specifies which command to run.

```
typedef enum e_can_cmd
{
    CAN_CMD_ABORT_TX,                // argument: transmit mailbox id
    CAN_CMD_RESET_TIMESTAMP,
    CAN_CMD_SET_MODE_COMM,           // start normal bus communications
    CAN_CMD_SET_MODE_TST_STANDARD,
    CAN_CMD_SET_MODE_TST_LISTEN,
    CAN_CMD_SET_MODE_TST_EXT_LOOPBACK,
    CAN_CMD_SET_MODE_TST_INT_LOOPBACK,
    CAN_CMD_END_ENUM
} can_cmd_t;
```

arg1

Argument which is specific to command. Most commands do not require an argument.

For the command CAN_CMD_ABORT_TX, the argument is a transmit mailbox id (see Section 2.10.1).

Return Values

CAN_SUCCESS:

Successful

CAN_ERR_INVALID_ARG:

An invalid argument was provided

CAN_ERR_ILLEGAL_MODE:

Changing to requested mode is illegal from current mode.

CAN_ERR_TIME_OUT:

Time Out error

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

This function is used for resetting the timestamp counter, aborting transmission of mailbox messages, and changing the CAN mode.

The following sequence of function calls is used to setup the CAN:

```
R_CAN_Open();
R_CAN_InitChan();
R_CAN_ConfigFIFO(); // do for 0 or more FIFOs
R_CAN_AddRxRule(); // do for 1-16 rules
```

Once the CAN is setup, the peripheral should enter normal communications mode or a test mode.

```
R_CAN_Control(); // Use CAN_CMD_SET_MODE_COMM or CAN_CMD_SET_MODE_TST_XXX
```

Note: If a Bus Off condition is detected on a channel, the channel enters Halt Mode and all communications cease. They cannot resume until after a Bus Off Recovery condition is detected and the application calls R_CAN_Control(CAN_CMD_SET_MODE_COMM).

Reentrant

Yes.

Example: Enter Normal Communications Mode

```
can_err_t err;  
  
err = R_CAN_Control(CAN_CMD_SET_MODE_COMM, 0);
```

Example: Abort Transmit

```
can_err_t err;  
  
/* Abort transmit on mailbox 0 on channel 0*/  
err = R_CAN_Control(CAN_CMD_ABORT_TX, CAN_BOX_CH0_TXMBX_0);
```

Special Notes:

Summary of different test modes:

- Standard Test Mode: Allows for CRC testing
- Listen-only Mode: Used for detecting communication speed. Cannot call R_CAN_SendMsg() in this mode.
- Internal Loopback Mode: Messages sent on a channel are handled as received messages and processed on that same channel. Here, the CAN transceiver is bypassed.
- External Loopback Mode: Same as Internal Loopback mode, only the transceiver is used.

R_CAN_SendMsg()

This function loads a message into a transmit mailbox or FIFO for transmission.

Format

```
can_err_t R_CAN_SendMsg(can_box_t box_id,
                        can_txmsg_t *p_txmsg);
```

Parameters

box_id

Transmit box id (mailbox or FIFO; see Section 2.10.1)

p_txmsg

Pointer to message to send

```
typedef struct st_can_txmsg
{
    uint8_t    ide;
    uint8_t    rtr;
    uint32_t    id;
    uint8_t    dlc;
    uint8_t    data[8];
    bool_t     one_shot;           // no retries on error; txmbx only
    bool_t     log_history;       // true if want to log
    uint8_t    label;             // 8-bit label for History FIFO
} can_txmsg_t;
```

Return Values

CAN_SUCCESS:

Successful

CAN_ERR_INVALID_ARG:

An invalid argument was provided

CAN_ERR_BOX_FULL:

Transmit mailbox or FIFO is full

CAN_ERR_ILLEGAL_MODE:

Cannot send message in current mode.

CAN_ERR_TIME_OUT:

Time Out error

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

This function places a message into a 1-message deep transmit mailbox or 4-message deep transmit FIFO. If there is already a message waiting to send in the mailbox, or 4 messages already exist in the FIFO, CAN_ERR_BOX_FULL is returned immediately. If the box_id is for a transmit mailbox and interrupts are not enabled (CAN_CFG_INT_MBX_TX_COMPLETE is 0), this function blocks until the message is sent. If interrupts are enabled or the message is for a transmit FIFO, the function will return immediately after loading the message into the transmit registers.

Reentrant

Yes, for different boxes.

Example:

```
can_txmsg_t    txmsg;
can_err_t      err;

/* Setup message */
txmsg.id = 0;                // ID field is 11-bits
txmsg.rtr = 0;              // local message
txmsg.id = 0x022;           // destination ID
txmsg.dlc = 5;              // data length
txmsg.data[0] = 'h';        // data...
txmsg.data[1] = 'e';
txmsg.data[2] = 'l';
txmsg.data[3] = 'l';
txmsg.data[4] = 'o';
txmsg.one_shot = false;     // do normal retries on error
txmsg.log_history = false;  // do not log in History FIFO
txmsg.label = 0;            // (label ignored because not logging message)

/*
 * Place message in transmit mailbox 2
 * If transmit complete interrupt is not enabled, the function returns
 *   after the message has been sent (assuming no error occurred).
 */
err = R_CAN_SendMsg(CAN_BOX_CH0_TXMBX_2, &txmsg);
```

Special Notes:

None.

R_CAN_GetMsg()

This function fetches a message from a receive mailbox or FIFO.

Format

```
can_err_t R_CAN_GetMsg(can_box_t box_id,
                      can_rxmsg_t *p_rxmsg);
```

Parameters

box_id

Receive box id (mailbox or FIFO; see Section 2.10.1)

p_rxmsg

Pointer to message buffer to load

```
typedef struct st_can_rxmsg
{
    uint8_t    ide;
    uint8_t    rtr;
    uint32_t    id;
    uint8_t    dlc;
    uint8_t    data[8];
    uint16_t    label;           // 12-bit label from receive rule
    uint16_t    timestamp;
} can_rxmsg_t;
```

Return Values

CAN_SUCCESS:

Successful

CAN_ERR_CH_NO_INIT:

Channel not initialized yet

CAN_ERR_INVALID_ARG:

An invalid argument was provided

CAN_ERR_BOX_EMPTY:

No message available to fetch

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

This function loads the message from a receive mailbox or FIFO into the message buffer provided. If there are no messages in the box, this function does not block and returns a CAN_ERR_BOX_EMPTY.

Reentrant

Yes, for different boxes.

Example:

```
can_rxmsg_t rxmsg;
can_err_t err;

/* Wait for message to appear in receive mailbox 3 */
while (R_CAN_GetMsg(CAN_BOX_RXMBX_3, &rxmsg) == CAN_ERR_BOX_EMPTY)
;

/* rxmsg contains message */
```

Special Notes:

None.

R_CAN_GetHistoryEntry()

This function fetches a log entry from a transmit history FIFO.

Format

```
can_err_t  R_CAN_GetHistoryEntry(can_box_t    box_id,  
                                can_history_t  *p_entry);
```

Parameters

box_id

Transmit history FIFO (see Section 2.10.1)

p_entry

Pointer to entry buffer to load

```
typedef struct st_can_history  
{  
    can_box_t    box_id;    // box which sent message  
    uint8_t      label;     // associated 8-bit label  
} can_history_t;
```

Return Values

CAN_SUCCESS:

Successful

CAN_ERR_INVALID_ARG:

An invalid argument was provided

CAN_ERR_BOX_EMPTY:

No entry available to fetch

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

An entry is added to the history FIFO each time an R_CAN_SendMsg() is called with the “log_history” in the argument structure is set to TRUE. This function loads a log entry from a transmit history FIFO into the entry buffer provided. If there are no entries in the FIFO, this function does not block and returns a CAN_ERR_BOX_EMPTY. The use of this feature is not required for normal operations.

Reentrant

Yes, for different boxes.

Example:

```
can_history_t  entry;  
can_err_t      err;  
  
/* Process all entries in transmit history FIFO */  
while (R_CAN_GetMsg(CAN_BOX_HIST_FIFO, &entry) == CAN_SUCCESS)  
{  
    /* process entries here */  
}
```

Special Notes:

None.

R_CAN_GetStatusMask()

This function returns a 32-bit mask based upon the status requested. Bit #defines have the form CAN_MASK_XXX.

Format

```
uint32_t R_CAN_GetStatusMask(can_stat_t type,
                             uint8_t chan,
                             can_err_t *p_err);
```

Parameters

type

Specifies which status to return.

```
typedef enum e_can_stat
{
    CAN_STAT_FIFO_EMPTY,
    CAN_STAT_FIFO_THRESHOLD,
    CAN_STAT_FIFO_OVFL,           // bits reset after reading
    CAN_STAT_RXMBX_FULL,
    CAN_STAT_GLOBAL_ERR,         // DLC error bit is reset after reading
    CAN_STAT_CH_TXMBX_SENT,      // bits reset after reading
    CAN_STAT_CH_TXMBX_ABORTED,   // bits reset after reading
    CAN_STAT_CH_ERROR,           // bits reset after reading
    CAN_STAT_END_ENUM
} can_stat_t;
```

chan

Specifies which channel to return status for (must be 0). Applies only to CAN_STAT_CH_XXX requests.

p_err

Pointer to returned error code.

CAN_SUCCESS: *Successful*
 CAN_ERR_INVALID_ARG: *An invalid argument was provided*

Return Values

32-bit box or error mask whose bit definitions have the form CAN_MASK_XXX and are defined in Section 2.10.10.

Properties

Prototyped in file "r_rscan_rx_if.h"

Description

This function returns a mask based upon the status type requested. All bit masks have the form CAN_MASK_XXX (see Section 2.10.10).

Reentrant

Yes.

Example

```
can_err_t err;
can_rxmsg_t rxmsg;

/* Wait for a message to come in on any receive mailbox */
while (R_CAN_GetStatusMask(CAN_STAT_RXMBX_FULL, 0, &err) == 0)
    ;

/* Check if receive mailbox 3 is full */
if (R_CAN_GetStatusMask(CAN_STAT_RXMBX_FULL, 0, &err) & CAN_MASK_RXMBX_3)
{
    /* get message */
    R_CAN_GetMsg(CAN_BOX_RXMBX_3, &rxmsg);
}
```

Special Notes:

None.

R_CAN_GetCountFIFO()

This function returns the number of items in a FIFO.

Format

```
uint32_t R_CAN_GetCountFIFO(can_box_t box_id,  
                             can_err_t *p_err);
```

Parameters

box_id

Specifies which FIFO to check (see Section 2.10.1).

p_err

Pointer to returned error code.

CAN_SUCCESS:

Successful

CAN_ERR_INVALID_ARG:

An invalid argument was provided

Return Values

Number of items in the FIFO

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

This function returns the number of items in the FIFO specified by *box_id*. This function is not required for normal operations.

Reentrant

Yes.

Example

```
uint32_t cnt;  
can_err_t err;  
  
/* Determine the number of messages in the History FIFO for channel 0 */  
cnt = R_CAN_GetCountFIFO(CAN_BOX_CH1_HIST_FIFO, &err);
```

Special Notes:

All FIFO usage is optional.

R_CAN_GetCountErr()

Returns the number of transmit or receive errors.

Format

```
uint32_t R_CAN_GetCountErr(can_count_t type,
                           uint8_t chan,
                           can_err_t *p_err);
```

Parameters

type

Specifies which status to return.

```
typedef enum e_can_count
{
    CAN_COUNT_RX_ERR,
    CAN_COUNT_TX_ERR,
    CAN_STAT_END_ENUM
} can_count_t;
```

chan

Specifies which channel to return error count for (must be 0).

p_err

Pointer to returned error code.

CAN_SUCCESS:

Successful

CAN_ERR_INVALID_ARG:

An invalid argument was provided

Return Values

The number of errors detected.

Properties

Prototyped in file "r_rscan_rx_if.h"

Description

This function returns the number of receive or transmit errors on a channel based upon the count type requested.

Reentrant

Yes.

Example

```
uint32_t rxcnt,txcnt;
can_err_t err;

/* Get the number of errors detected */
rxcnt = R_CAN_GetCountErr(CAN_COUNT_RX_ERR, CAN_CH0, &err);
txcnt = R_CAN_GetCountErr(CAN_COUNT_TX_ERR, CAN_CH0, &err);
```

Special Notes:

This use of this function is optional. It can be used to detect the health of the network and how close the network is to entering the Error Passive state (128 errors) or Bus Off state (255 errors).

R_CAN_Close()

This function removes clock from the CAN peripheral and disables the associated interrupts.

Format

```
void R_CAN_Close(void);
```

Parameters

None

Return Values

None

Properties

Prototyped in file “r_scan_rx_if.h”

Description

This function halts all existing communications, disables all interrupts (if any), and shuts down the peripheral.

Reentrant

Yes, but no need to ever call more than once.

Example

```
R_CAN_Close();
```

Special Notes:

None.

R_CAN_GetVersion()

This function returns the driver version number at runtime.

Format

```
uint32_t R_CAN_GetVersion(void);
```

Parameters

None

Return Values

Version number.

Properties

Prototyped in file “r_rscan_rx_if.h”

Description

Returns the version of this module. The version number is encoded such that the top two bytes are the major version number and the bottom two bytes are the minor version number.

4. Pin Setting

To use the RSCAN FIT module, assign input/output signals of the peripheral function to pins with the multi-function pin controller (MPC). The pin assignment is referred to as the “Pin Setting” in this document. Please perform the pin setting after calling the R_CAN_Open function.

5. Demo Project

Demo projects are complete stand-alone programs. They include function main() that utilizes the module and its dependent modules (e.g., r_bsp). The standard naming convention for the demo project is <module>_demo_<board> where <module> is the peripheral acronym (e.g. s12ad, cmt, sci) and the <board> is the standard RSK (e.g. rskrx231). For example, rscan FIT module demo project for RSKRX231 will be named as rscan_demo_rskrx231. Similarly, the exported .zip file will be <module>_demo_<board>.zip. For the same example, the zipped export/import file will be named as rscan_demo_rskrx231.zip.

5.1 rscan_demo_rskrx231, rscan_demo_rskrx231_gcc

This program requires the connection of a CAN device (such as a sniffer) capable of receiving and sending messages. The program spins in a loop sending then receiving one message at a time. The messages received must have an ID of 0x60-0x6F and contain at least 4 bytes of data.

The baud rate is set to 500Kbps.

This program can run using either mailboxes without interrupts or FIFOs with interrupts. The desired operation is configured by changing the value of USE_FIFOS in main.c to 0 for mailboxes or 1 for FIFOs.

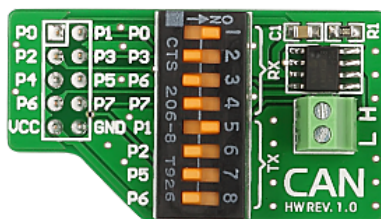
5.2 rscan_demo_rskrx24t, rscan_demo_rskrx24t_gcc

This program requires the connection of a CAN device (such as a sniffer) capable of receiving and sending messages. The program spins in a loop sending then receiving one message at a time. The messages received must have an ID of 0x60-0x6F and contain at least 4 bytes of data.

The baud rate is set to 500Kbps.

This program can run using either mailboxes without interrupts or FIFOs with interrupts. The desired operation is configured by changing the value of USE_FIFOS in main.c to 0 for mailboxes or 1 for FIFOs.

The RSKRX24T must be populated with a large memory version of the RX24T (such as the 512Kb EAxFP) in order to have CAN peripheral support. Additionally, an external CAN transceiver board is required. The following is an example using the MikroElektronika CAN-1 board (www.mikroe.com/add-on-boards/communication/can).

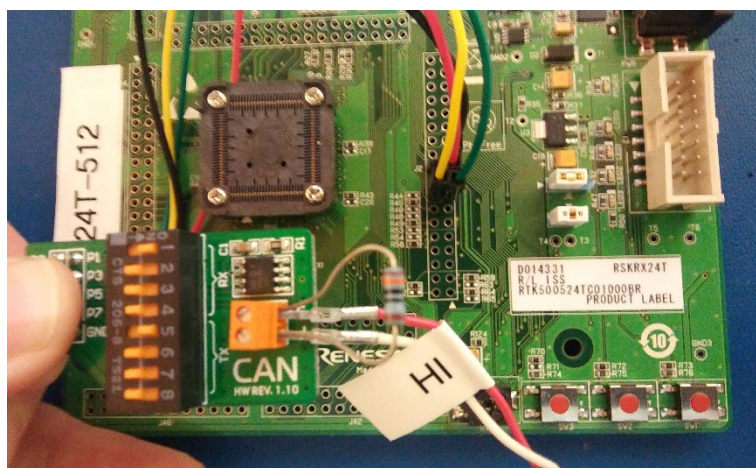
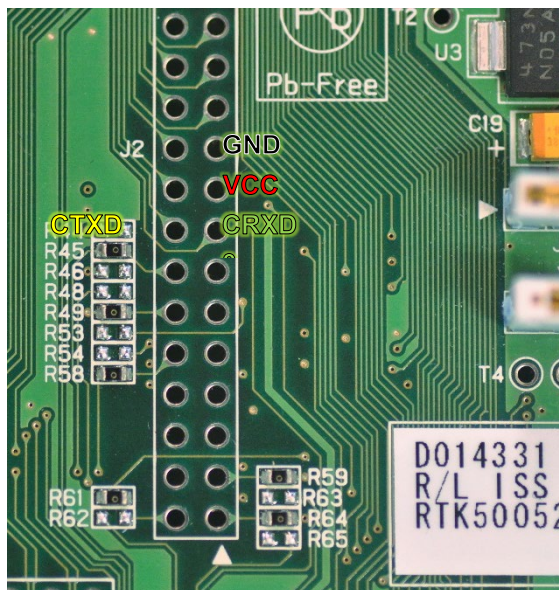


It is recommended to solder a 2x3 header across J2 pins 15-20 on the RSKRX24T. Dip switch lines 1 and 5 should be in the ON position on the CAN-1 board. Connect the boards in the following manner:

	RSKRX24T	CAN-1
CRXD0	J2 pin 15	P0
CTXD0	J2 pin 16	P1
VCC	J2 pin 17	VCC
GND	J2 pin 19	GND

120Ω resistor
network)

P0-P1 (opt. termination resistor depending upon your



5.3 rscan_demo_rskrx24u, rscan_demo_rskrx24u_gcc

This program requires the connection of a CAN device (such as a sniffer) capable of receiving and sending messages. The program spins in a loop sending then receiving one message at a time. The messages received must have an ID of 0x60-0x6F and contain at least 4 bytes of data.

The baud rate is set to 500Kbps.

This program can run using either mailboxes without interrupts or FIFOs with interrupts. The desired operation is configured by changing the value of USE_FIFOS in main.c to 0 for mailboxes or 1 for FIFOs.

5.4 rscan_demo_rskrx140, rscan_demo_rskrx140_gcc

This program requires the connection of a CAN device (such as a sniffer) capable of receiving and sending messages. The program spins in a loop sending then receiving one message at a time. The messages received must have an ID of 0x60-0x6F and contain at least 4 bytes of data.

The baud rate is set to 500Kbps.

This program can run using either mailboxes without interrupts or FIFOs with interrupts. The desired operation is configured by changing the value of USE_FIFOS in main.c to 0 for mailboxes or 1 for FIFOs.

6. Appendices

6.1 Confirmed Operation Environment

This section describes confirmed operation environment for the RSCAN FIT module.

Table 6.1 Confirmed Operation Environment (Rev.2.93)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2025-12 IAR Embedded Workbench for Renesas RX 5.20.1
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.07.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 14.02.00.202511 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 5.20.1 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.93
Board used	-

Table 6.2 Confirmed Operation Environment (Rev.2.92)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2025-01 IAR Embedded Workbench for Renesas RX 5.10.1
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.07.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202411 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 5.10.1 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.92
Board used	-

Table 6.3 Confirmed Operation Environment (Rev.2.91)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2025-01 IAR Embedded Workbench for Renesas RX 5.10.1
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.07.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202411 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 5.10.1 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.91
Board used	-

Table 6.4 Confirmed Operation Environment (Rev.2.90)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2024-07 IAR Embedded Workbench for Renesas RX 5.10.1
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.06.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202405 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 5.10.1 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.90
Board used	Renesas Solution Starter Kit for RX23E-B (product No.: RTK0ES1001C00001BJ)

Table 6.5 Confirmed Operation Environment (Rev.2.80)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2023-10 IAR Embedded Workbench for Renesas RX 4.20.3
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.05.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202305 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.20.3 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.80
Board used	Renesas Solution Starter Kit for RX23E-B (product No.: RTK0ES1001C00001BJ)

Table 6.6 Confirmed Operation Environment (Rev.2.70)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2023-04 IAR Embedded Workbench for Renesas RX 4.20.3
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.05.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202204 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.20.3 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.70
Board used	Renesas Solution Starter Kit for RX23E-B (product No.: RTK0ES1001C00001BJ)

Table 6.7 Confirmed Operation Environment (Rev.2.60)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2023-01 IAR Embedded Workbench for Renesas RX 4.20.3
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.05.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202204 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.20.3 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.60
Board used	Renesas Starter Kit for RX140 (product No.: RTK551406BCxxxxxx)

Table 6.8 Confirmed Operation Environment (Rev.2.50)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2022-07 IAR Embedded Workbench for Renesas RX 4.20.3
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.04.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202202 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.20.3 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.50
Board used	Renesas Starter Kit+ for RX231 (product No.: R0K505231SxxxBE) Renesas Starter Kit+ for RX24U (product No.: RTK500524UCxxxxxBR)

Table 6.9 Confirmed Operation Environment (Rev.2.40)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 2022-01 IAR Embedded Workbench for Renesas RX 4.20.3
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.04.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.202104 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.20.3 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.40
Board used	Renesas Starter Kit+ for RX140 (product No.: RTK55140xxxxxxxxxx)

Table 6.10 Confirmed Operation Environment (Rev.2.30)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 7.8.0
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.02.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.201904 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module
Endian	Little endian
Revision of the module	Rev.2.30
Board used	Renesas Starter Kit+ for RX231 (product No.: RTK505231xxxxxxxx) Renesas Starter Kit+ for RX24U (product No.: RTK50524Uxxxxxxxx)

Table 6.11 Confirmed Operation Environment (Rev.2.21)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 7.7.0 IAR Embedded Workbench for Renesas RX 4.12.1
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.02.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 8.3.0.201904 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.12.1 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.21
Board used	Renesas Solution Starter Kit+ for RX23E-A (product No.: RTK0ESXBxxxxxxxxxx)

Table 6.12 Confirmed Operation Environment (Rev.2.20)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 7.7.0 IAR Embedded Workbench for Renesas RX 4.12.1
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.01.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 4.8.4.201902 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.12.1 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.20
Board used	Renesas Starter Kit+ for RX23E-A (product No.: RTK5523E-Axxxxxxxxxx)

Table 6.13 Confirmed Operation Environment (Rev.2.10)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 7.5.0
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.01.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99
Endian	Big endian/little endian
Revision of the module	Rev.2.10
Board used	Renesas Starter Kit+ for RX23W (product No.: RTK5523Wxxxxxxx)

Table 6.14 Confirmed Operation Environment (Rev.2.00)

Item	Contents
Integrated development environment	Renesas Electronics e ² studio Version 7.4.0 IAR Embedded Workbench for Renesas RX 4.10.1
C compiler	Renesas Electronics C/C++ Compiler Package for RX Family V3.01.00 Compiler option: The following option is added to the default settings of the integrated development environment. -lang = c99 GCC for Renesas RX 4.8.4.201803 Compiler option: The following option is added to the default settings of the integrated development environment. -std=gnu99 Linker option: The following user defined option should be added to the default settings of the integrated development environment, if "Optimize size (-Os)" is used: -Wl,--no-gc-sections This is to work around a GCC linker issue whereby the linker erroneously discard interrupt functions declared in FIT peripheral module IAR C/C++ Compiler for Renesas RX version 4.10.1 Compiler option: The default settings of the integrated development environment.
Endian	Big endian/little endian
Revision of the module	Rev.2.00
Board used	Renesas Starter Kit for RX231 (product No.: R0K505231xxxxxx)

6.2 Troubleshooting

(1) Q: I have added the FIT module to the project and built it. Then I got the error: Could not open source file "platform.h".

A: The FIT module may not be added to the project properly. Check if the method for adding FIT modules is correct with the following documents:

- Using CS+:

Application note "Adding Firmware Integration Technology Modules to CS+ Projects (R01AN1826)"

- Using e² studio:

Application note "Adding Firmware Integration Technology Modules to Projects (R01AN1723)"

When using a FIT module, the board support package FIT module (BSP module) must also be added to the project. Refer to the application note "Board Support Package Module Using Firmware Integration Technology (R01AN1685)".

(2) Q: I have added the FIT module to the project and built it. Then I got the error: This MCU is not supported by the current r_rscan_rx module.

A: The FIT module you added may not support the target device chosen in your project. Check the supported devices of added FIT modules.

(3) Q: I have added the FIT module to the project and built it. Then I got an error for when the configuration setting is wrong.

A: The setting in the file "r_rscan_rx_config.h" may be wrong. Check the file "r_rscan_rx_config.h". If there is a wrong setting, set the correct value for that. Refer to 2.8 Configuration Overview for details.

Revision History

Rev.	Date	Description	
		Page	Summary
1.00	May 20, 2015	—	Initial release
1.10	Sep 12, 2016	1,4,5,17, 18	Added support for RX230, RX24U, and RX24T-512
1.20	Dec 06, 2018	—	Fixed big endian bug. Added __evenaccess to private structure and union definitions.
1.21	Feb 01, 2019	—	Changes associated with functions: Added support setting function of configuration option Using GUI on Smart Configurator. [Description] Added a setting file to support configuration option setting function by GUI.
2.00	May.20.19	— 1 5 7 37 38 40 40 40 program	Supported the following compilers: - GCC for Renesas RX - IAR C/C++ Compiler for Renesas RX Target Devices: Removed 512KB from RX24T group Added the section of Target compilers. Deleted related documents. 2.3 Software Requirements Requires r_bsp v5.20 or higher Updated the section of 2.9 Code Size. Added 4. Pin Setting Changed section “5. Demo Projects” Deleted the section of Website and Support. Added 6. Appendices 6.1 Confirmed Operation Environment: Added table for Rev.2.00 Changed below for support GCC and IAR compiler: 1. Deleted the inline expansion of the R_CAN_GetVersion function. 2. Replaced nop with the intrinsic functions of BSP. 3. Replaced the declaration of interrupt functions with the macro definition of BSP.
2.10	Jun.28.19	1 8 40 Program	Added support for RX23W Added code size corresponding to RX23W 6.1 Confirmed Operation Environment: Added Table for Rev.2.10 Added support for RX23W.
2.20	Oct.15.19	1 8 10, 23, 24 40 Program	Added support for RX23E-A Added code size corresponding to RX23E-A Changed data type of check_ide and check_rtr 6.1 Confirmed Operation Environment: Added Table for Rev.2.20 Added support for RX23E-A. Added support for atomic control Fixed IAR warnings in R_CAN_GetMsg() Changed data type of check_ide and check_rtr
2.21	Mar.31.20	8 41	Updated the section of 2.9 Code Size 6.1 Confirmed Operation Environment: Added Table for Rev.2.21

Rev.	Date	Description	
		Page	Summary
2.30	Jun.30.20	39,40	Updated and added new demo project
		41	6.1 Confirmed Operation Environment: Added Table for Rev.2.30
		Program	Updated and added new demo project
2.31	Oct.30.20	Program	Made correction to XML <zipsource>
2.32	Sep.13.21	19	Removed reference to APN R01AN3455.
		Program	Added CS+ support for demo project.
2.40	Nov.11.21	1, 4	Added support for RX140 (products with 128-Kbyte or larger ROM)
		8	Updated the section of 2.9 Code Size
		41	6.1 Confirmed Operation Environment: Added Table for Rev.2.40
		Program	Added support for RX140 (products with 128-Kbyte or larger ROM)
2.41	Jun.30.22	21	Remove CAN_ERR_MAX_ONE_GWFIFO macro from application note
		Program	Changed the default value of CAN clock source to PCLK.
2.50	Jul.29.22	41	Table 6.1: Confirm Operation Environment: Added Table for Rev. 2.50.
		9	Fixed RXMBX enum error: CAN_BOX_RXMBX_4 is removed and replaced by CAN_BOX_RXMBX_3. Changed value of CAN_BOX_RXMBX_2
		Program	Updated demo projects Fixed RXMBX enum error: CAN_BOX_RXMBX_4 is removed and replaced by CAN_BOX_RXMBX_3. Changed value of CAN_BOX_RXMBX_2
2.60	Mar.21.2023	41	Updated and added new demo project
		42	6.1 Confirmed Operation Environment: Added Table for Rev.2.60
		Program	Added new demo project
2.70	May.29.2023	1	Added support for RX23E-B
		8	Added code size corresponding to RX23E-B
		10	Fixed fifo threshold enum error.
		13	Deleted the description of FIT configurator and updated new description in "2.12 Adding the FIT Module to Your Project"
		14	Added section 2.13 "for", "while", and "do while" statements.
		42	6.1 Confirmed Operation Environment: Added Table for Rev.2.70
		Program	Added support for RX23E-B. Fixed fifo threshold enum error.
2.80	Nov.17.2023	13	2.11 Return Values: Added CAN_ERR_TIME_OUT return value.
		17, 27, 29	R_CAN_Open(), R_CAN_Control(), and R_CAN_SendMsg(): - Return Values: Added CAN_ERR_TIME_OUT.
		43	6.1 Confirmed Operation Environment: Added Table for Rev.2.80
		Program	Added CAN_ERR_TIME_OUT return to while loop in R_CAN_Open, R_CAN_Control, and R_CAN_SendMsg function. Added WAIT_LOOP comments.

Rev.	Date	Description	
		Page	Summary
2.90	Dec.31.2024	1	Added support for RX24T Version B. Removed support for RX230.
		7	Added new macros CAN_CFG_EN_NESTED_INT to support nested interrupt.
		43	6.1 Confirmed Operation Environment: Added Table for Rev.2.90
		Program	Added support Nested Interrupt. Added description of supported RX231. Modified code to support only RX24T chip version B. Removed support for RX230.
2.91	Mar.15.2025	43	6.1 Confirmed Operation Environment: Added Table for Rev.2.91
		Program	Updated FIT Disclaimer and Copyright.
2.92	Oct.30.2025	43	6.1 Confirmed Operation Environment: Added Table for Rev.2.92
		Program	Removed doc folder and updated .rcpc file in FITDemos.
2.93	Mar.30.2026	43	6.1 Confirmed Operation Environment: Added Table for Rev.2.93
		Program	Included spdx file in the package.

General Precautions in the Handling of Microprocessing Unit and Microcontroller Unit Products

The following usage notes are applicable to all Microprocessing unit and Microcontroller unit products from Renesas. For detailed usage notes on the products covered by this document, refer to the relevant sections of the document as well as any technical updates that have been issued for the products.

1. Precaution against Electrostatic Discharge (ESD)

A strong electrical field, when exposed to a CMOS device, can cause destruction of the gate oxide and ultimately degrade the device operation. Steps must be taken to stop the generation of static electricity as much as possible, and quickly dissipate it when it occurs. Environmental control must be adequate. When it is dry, a humidifier should be used. This is recommended to avoid using insulators that can easily build up static electricity.

Semiconductor devices must be stored and transported in an anti-static container, static shielding bag or conductive material. All test and measurement tools including work benches and floors must be grounded. The operator must also be grounded using a wrist strap. Semiconductor devices must not be touched with bare hands. Similar precautions must be taken for printed circuit boards with mounted semiconductor devices.

2. Processing at power-on

The state of the product is undefined at the time when power is supplied. The states of internal circuits in the LSI are indeterminate and the states of register settings and pins are undefined at the time when power is supplied. In a finished product where the reset signal is applied to the external reset pin, the states of pins are not guaranteed from the time when power is supplied until the reset process is completed. In a similar way, the states of pins in a product that is reset by an on-chip power-on reset function are not guaranteed from the time when power is supplied until the power reaches the level at which resetting is specified.

3. Input of signal during power-off state

Do not input signals or an I/O pull-up power supply while the device is powered off. The current injection that results from input of such a signal or I/O pull-up power supply may cause malfunction and the abnormal current that passes in the device at this time may cause degradation of internal elements. Follow the guideline for input signal during power-off state as described in your product documentation.

4. Handling of unused pins

Handle unused pins in accordance with the directions given under handling of unused pins in the manual. The input pins of CMOS products are generally in the high-impedance state. In operation with an unused pin in the open-circuit state, extra electromagnetic noise is induced in the vicinity of the LSI, an associated shoot-through current flows internally, and malfunctions occur due to the false recognition of the pin state as an input signal become possible.

5. Clock signals

After applying a reset, only release the reset line after the operating clock signal becomes stable. When switching the clock signal during program execution, wait until the target clock signal is stabilized. When the clock signal is generated with an external resonator or from an external oscillator during a reset, ensure that the reset line is only released after full stabilization of the clock signal. Additionally, when switching to a clock signal produced with an external resonator or by an external oscillator while program execution is in progress, wait until the target clock signal is stable.

6. Voltage application waveform at input pin

Waveform distortion due to input noise or a reflected wave may cause malfunction. If the input of the CMOS device stays in the area between V_{IL} (Max.) and V_{IH} (Min.) due to noise, for example, the device may malfunction. Take care to prevent chattering noise from entering the device when the input level is fixed, and also in the transition period when the input level passes through the area between V_{IL} (Max.) and V_{IH} (Min.).

7. Prohibition of access to reserved addresses

Access to reserved addresses is prohibited. The reserved addresses are provided for possible future expansion of functions. Do not access these addresses as the correct operation of the LSI is not guaranteed.

8. Differences between products

Before changing from one product to another, for example to a product with a different part number, confirm that the change will not lead to problems. The characteristics of a microprocessing unit or microcontroller unit products in the same group but having a different part number might differ in terms of internal memory capacity, layout pattern, and other factors, which can affect the ranges of electrical characteristics, such as characteristic values, operating margins, immunity to noise, and amount of radiated noise. When changing to a product with a different part number, implement a system-evaluation test for the given product.

Notice

1. Descriptions of circuits, software and other related information in this document are provided only to illustrate the operation of semiconductor products and application examples. You are fully responsible for the incorporation or any other use of the circuits, software, and information in the design of your product or system. Renesas Electronics disclaims any and all liability for any losses and damages incurred by you or third parties arising from the use of these circuits, software, or information.
2. Renesas Electronics hereby expressly disclaims any warranties against and liability for infringement or any other claims involving patents, copyrights, or other intellectual property rights of third parties, by or arising from the use of Renesas Electronics products or technical information described in this document, including but not limited to, the product data, drawings, charts, programs, algorithms, and application examples.
3. No license, express, implied or otherwise, is granted hereby under any patents, copyrights or other intellectual property rights of Renesas Electronics or others.
4. You shall be responsible for determining what licenses are required from any third parties, and obtaining such licenses for the lawful import, export, manufacture, sales, utilization, distribution or other disposal of any products incorporating Renesas Electronics products, if required.
5. You shall not alter, modify, copy, or reverse engineer any Renesas Electronics product, whether in whole or in part. Renesas Electronics disclaims any and all liability for any losses or damages incurred by you or third parties arising from such alteration, modification, copying or reverse engineering.
6. Renesas Electronics products are classified according to the following two quality grades: "Standard" and "High Quality". The intended applications for each Renesas Electronics product depends on the product's quality grade, as indicated below.

"Standard": Computers; office equipment; communications equipment; test and measurement equipment; audio and visual equipment; home electronic appliances; machine tools; personal electronic equipment; industrial robots; etc.

"High Quality": Transportation equipment (automobiles, trains, ships, etc.); traffic control (traffic lights); large-scale communication equipment; key financial terminal systems; safety control equipment; etc.

Unless expressly designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not intended or authorized for use in products or systems that may pose a direct threat to human life or bodily injury (artificial life support devices or systems; surgical implantations; etc.), or may cause serious property damage (space system; undersea repeaters; nuclear power control systems; aircraft control systems; key plant systems; military equipment; etc.). Renesas Electronics disclaims any and all liability for any damages or losses incurred by you or any third parties arising from the use of any Renesas Electronics product that is inconsistent with any Renesas Electronics data sheet, user's manual or other Renesas Electronics document.

7. No semiconductor product is absolutely secure. Notwithstanding any security measures or features that may be implemented in Renesas Electronics hardware or software products, Renesas Electronics shall have absolutely no liability arising out of any vulnerability or security breach, including but not limited to any unauthorized access to or use of a Renesas Electronics product or a system that uses a Renesas Electronics product. RENASAS ELECTRONICS DOES NOT WARRANT OR GUARANTEE THAT RENASAS ELECTRONICS PRODUCTS, OR ANY SYSTEMS CREATED USING RENASAS ELECTRONICS PRODUCTS WILL BE INVULNERABLE OR FREE FROM CORRUPTION, ATTACK, VIRUSES, INTERFERENCE, HACKING, DATA LOSS OR THEFT, OR OTHER SECURITY INTRUSION ("Vulnerability Issues"). RENASAS ELECTRONICS DISCLAIMS ANY AND ALL RESPONSIBILITY OR LIABILITY ARISING FROM OR RELATED TO ANY VULNERABILITY ISSUES. FURTHERMORE, TO THE EXTENT PERMITTED BY APPLICABLE LAW, RENASAS ELECTRONICS DISCLAIMS ANY AND ALL WARRANTIES, EXPRESS OR IMPLIED, WITH RESPECT TO THIS DOCUMENT AND ANY RELATED OR ACCOMPANYING SOFTWARE OR HARDWARE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE.
8. When using Renesas Electronics products, refer to the latest product information (data sheets, user's manuals, application notes, "General Notes for Handling and Using Semiconductor Devices" in the reliability handbook, etc.), and ensure that usage conditions are within the ranges specified by Renesas Electronics with respect to maximum ratings, operating power supply voltage range, heat dissipation characteristics, installation, etc. Renesas Electronics disclaims any and all liability for any malfunctions, failure or accident arising out of the use of Renesas Electronics products outside of such specified ranges.
9. Although Renesas Electronics endeavors to improve the quality and reliability of Renesas Electronics products, semiconductor products have specific characteristics, such as the occurrence of failure at a certain rate and malfunctions under certain use conditions. Unless designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not subject to radiation resistance design. You are responsible for implementing safety measures to guard against the possibility of bodily injury, injury or damage caused by fire, and/or danger to the public in the event of a failure or malfunction of Renesas Electronics products, such as safety design for hardware and software, including but not limited to redundancy, fire control and malfunction prevention, appropriate treatment for aging degradation or any other appropriate measures. Because the evaluation of microcomputer software alone is very difficult and impractical, you are responsible for evaluating the safety of the final products or systems manufactured by you.
10. Please contact a Renesas Electronics sales office for details as to environmental matters such as the environmental compatibility of each Renesas Electronics product. You are responsible for carefully and sufficiently investigating applicable laws and regulations that regulate the inclusion or use of controlled substances, including without limitation, the EU RoHS Directive, and using Renesas Electronics products in compliance with all these applicable laws and regulations. Renesas Electronics disclaims any and all liability for damages or losses occurring as a result of your noncompliance with applicable laws and regulations.
11. Renesas Electronics products and technologies shall not be used for or incorporated into any products or systems whose manufacture, use, or sale is prohibited under any applicable domestic or foreign laws or regulations. You shall comply with any applicable export control laws and regulations promulgated and administered by the governments of any countries asserting jurisdiction over the parties or transactions.
12. It is the responsibility of the buyer or distributor of Renesas Electronics products, or any other party who distributes, disposes of, or otherwise sells or transfers the product to a third party, to notify such third party in advance of the contents and conditions set forth in this document.
13. This document shall not be reprinted, reproduced or duplicated in any form, in whole or in part, without prior written consent of Renesas Electronics.
14. Please contact a Renesas Electronics sales office if you have any questions regarding the information contained in this document or Renesas Electronics products.

(Note1) "Renesas Electronics" as used in this document means Renesas Electronics Corporation and also includes its directly or indirectly controlled subsidiaries.

(Note2) "Renesas Electronics product(s)" means any product developed or manufactured by or for Renesas Electronics.

(Rev.5.0-1 October 2020)

Corporate Headquarters

TOYOSU FORESIA, 3-2-24 Toyosu,
Koto-ku, Tokyo 135-0061, Japan
www.renesas.com

Trademarks

Renesas and the Renesas logo are trademarks of Renesas Electronics Corporation. All trademarks and registered trademarks are the property of their respective owners.

Contact information

For further information on a product, technology, the most up-to-date version of a document, or your nearest sales office, please visit:
www.renesas.com/contact/.